

Traffic pattern between M2M

Traffic pattern between M2M in major IoT applications can be mainly modelled into three types: periodic update, event-driven and payload exchange.

Periodic update is used in non-real-time and event-triggered applications at regular time patterns. Examples are smart meter readings such as cooking gas, electricity and water.

Event-driven is used in real time or non real time, based on event monitoring. Examples include alarms and emergency alerts.

Payload exchange is used in response to the previous periodic update or event-driven traffic. This model shows that most IoT applications do not have continuous traffic all the time. This allows designers to think about using power-saving methods efficiently at protocol or architectural level.

Most architecture- or protocol-level implementations based on standard connectivity protocols comprise active states (with and without traffic) and low-power states. It is common to implement payload exchange to save power during protocol-defined low-power states. But during active states with minimal or no traffic, payload exchange cannot be used, because latencies in switching off power and powering it back on often exceed protocol timeouts for entry and exit. Therefore to reduce power during active state, clock gating is used at architectural level. This can be achieved based on traffic pattern.

For transfers that are carried out in bursts, it is possible to enter low-power state when there is no traffic and save power. But in case of periodic transfers, the device needs to remain in active state. When it is in active state for applications such as tracking/monitoring systems, traffic may not be present for the complete period.

Clock gating can be used effectively to save power when there is no traffic, if implemented prudently. In this approach, instead of relying on protocol-specific low-power states to perform clock gating, efficient ways

for specific blocks are defined based on the internal architecture.

Consider USB2.0 protocol as an example. It comprises active states (with and without traffic) and low-power modes such as L1/L2 (sleep/suspend). During low-power mode, USB sub-system (controller and physical layer) can enter physical layer or clock gating depending on entry and exit duration. When USB sub-system is in active state, physical layer power is 20 to 50 times controller power.

Traditionally, efforts have been made to reduce physical layer active power. But, there is a fundamental limit beyond which it is not easy to reduce physical layer power during active traffic.

Efforts have also been made on reducing physical layer power when controller/physical layer sub-system is not functionally used in the system and during suspend conditions. During L2 (long duration of entry and exit), amount of power consumed by the sub-system can be very low due to controller power gating and physical layer low-power techniques. But many use cases cannot make use of USB L2. Operating system support for USB L2 has been slow.

Transition time between active and suspend is often 100+ milliseconds in practice. When USB applications require much lesser response transition time than L2, these take advantage of L1. During L1, physical layer power can be reduced by more than 99 per cent by appropriate choice of active/idle transition times. But controller sleep power is not significantly reduced with traditional clock gating.

A controller has limited power-saving capability due to its requirement to wake up fast from sleep. Hence, it cannot power gate. In practice, this means a controller's sleep power is 10 to 20 times the physical layer sleep power. Clock gating handles power saving when there is no traffic and during low-power modes.

Clock gating methods or traffic pattern between M2M.

1. EDA TIGG (register level)
2. Architectural clock gating

EDA TIGG. Currently, EDA tools provide clock gating options during synthesis. This is an efficient way of inserting register-level clock gating cells with minimal effort. The tool, based on RTL coding style, detects synchronous load-enable registers (flip-flops that share the same clock and synchronous control signals) and inserts clock gating cells with synchronous control signals as enable condition to the corresponding clock.

For Design Compiler tool, automatic insertion of clock gating is done. For example, if RTL is coded as below, DC checks the synchronous load enable signal `clk_en` in all possible flops and then uses `clk_en` as one of the enable conditions to gate corresponding flops.

```
always@(posedge clk or negedge reset)
begin
if (reset == 1'b0) // asynchronous reset
q <= 32'd0;
else
if (clk_en) // synchronous load enable
q <= d;
end
```

This method can be used for all existing IPs. It provides optimal power saving to IPs that have a large register count.

First, synthesise the IP and then analyse register count. If there is a large register count, reduce large dynamic power. If there is synchronous load enable(s) in the design, analyse gated and ungated register reports in the IP. If possible, check ungated registers to see if there is a way to change RTL coding style so that the tool can gate it. Manually estimate the power saved in the IP for gating certain registers using the tool. Compare this by changing all ungated registers with gated registers.

Using this method, in most cases, efforts involved in changing ungated register to gated register is high. In some cases, it is not easy to modify the coding style, while some registers (CDC synchronisers) may not have synchronous load enables. Therefore power saving from TIGG does not give a completely optimal value, but there can be further power reduction in all cases. **EFY**